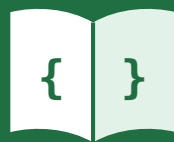

Module 20 — Docker and Deployment

Exercises

Spring Boot Edu



2026-09-25

Contents

How to Work	2
Exercise 1 — Optimize the Dockerfile (Medium)	2
Exercise 2 — A System with Compose (Medium)	2
Exercise 3 — Readiness Depends on the Warehouse (Medium)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/20-docker-deployment/exercise/`. Find the TODO comments (also in `Dockerfile` and `compose.yaml`).
2. The tests build Docker images, so they are integration tests (*IT) and need the jar: run them with `verify`, not `test`. Docker must be running.

```
./mvnw -Pexercises -pl modules/20-docker-deployment/exercise -am verify
```

3. The exercise is done when all tests are green.
4. If you get stuck, read the hints first, then the solution in `modules/20-docker-deployment/solution/`.

Exercise 1 — Optimize the Dockerfile (Medium)

Goal: `exercise/Dockerfile` works, but the image is about 400 MB, contains a full JDK, runs as root and puts the whole jar into one layer.

Tasks (`exercise/Dockerfile`):

- TODO 1a — a multi-stage build: extract the jar into layers and copy them into the final image one by one.
- TODO 1b — the final image contains a JRE built with `jdeps` and `jlink`, not a JDK.
- TODO 1c — run as a user that is not root.
- TODO 1d — use the `exec` form, so that `java` is PID 1.

Hints:

- Lesson section 3.2 shows every step. Try to write it yourself first, then compare.
- `jlink --strip-debug` on Alpine needs `apk add binutils`.
- `Exercise1IT` checks the user (`id -u`), PID 1 (`/proc/1/cmdline`), the size (`< 150 MB`) and that there is no `javac`.
- The PID 1 test is already green: `BusyBox sh -c` replaces itself with a single command. The `exec` form is still the reliable way (it works with every shell and with several commands).

Acceptance criteria: all 4 tests in `Exercise1IT` pass.

Estimated time: 40 minutes

Exercise 2 — A System with Compose (Medium)

Goal: the bookstore needs its database and a warehouse service. `exercise/compose.yaml` describes the system only partly.

Tasks (exercise/compose.yaml):

- TODO 2a — a healthcheck for PostgreSQL (pg_isready).
- TODO 2b — a service warehouse from nginx:1.29-alpine.
- TODO 2c — configure the app with environment variables: datasource URL, user, password, and BOOKSTORE_WAREHOUSE_HOST / BOOKSTORE_WAREHOUSE_PORT.
- TODO 2d — start the app only when the database is healthy and the warehouse has started.

Hints:

- Lesson section 3.6: condition: service_healthy and condition: service_started.
- The services reach each other by service name: jdbc:postgresql://postgres:5432/bookstore, warehouse port 80.
- Exercise2IT starts your compose file and waits up to 3 minutes for /actuator/health/readiness. Try it by hand first: `docker compose -f modules/20-docker-deployment/exercise/compose.yaml up --build`.

Acceptance criteria: both tests in Exercise2IT pass.

Estimated time: 30 minutes

Exercise 3 — Readiness Depends on the Warehouse (Medium)

Goal: without the warehouse, orders cannot be delivered, so the application should receive no traffic. But restarting it would not bring the warehouse back.

Tasks:

- TODO 3a (warehouse.WarehouseHealthIndicator) — UP when a TCP connection to the warehouse works (timeout 500 ms), otherwise DOWN with the exception. Add the address as a detail.
- TODO 3b (application.yaml) — add the health component warehouse to the readiness group, not to liveness.

Hints:

- `new Socket().connect(new InetSocketAddress(host, port), 500)` in a try-with-resources.
- `Health.up().withDetail(...).build()` and `Health.down().withException(e).build()`.
- The component name comes from the bean name without HealthIndicator: warehouse.

Acceptance criteria: Exercise3IT passes.

Estimated time: 25 minutes

Extra Challenge (Optional)

Add a HEALTHCHECK to your Dockerfile from exercise 1 that uses the readiness probe, and change `depends_on` of a second `app` instance in `compose` to wait for the first one with `condition: service_healthy`. Then stop the warehouse (`docker compose stop warehouse`) and watch `docker compose ps`.